

Методические указания для студентов

Практическое занятие 3.3.2 (ПЗ-10)

Реализация упрощённого offline/online-пайплайна обработки данных и автоматизация вычислительного процесса

1. Общие положения

Практическое занятие 3.3.2 посвящено не отдельной операции анализа данных, а построению учебной модели промышленного пайплайна обработки данных. Это означает, что в центре внимания находится не только итоговый результат, но и сам путь, который проходят данные от момента загрузки до формирования аналитической витрины.

В рамках этой работы студент должен увидеть, что современная обработка больших данных – это не бессистемный набор команд в Jupyter Notebook и не разовый расчёт показателей. Реальная аналитическая система включает несколько логически связанных этапов: получение данных, их очистку, преобразование, объединение источников, агрегирование, построение итогового результата, а в более зрелом виде – ещё и автоматизацию, контроль и оркестрацию. Именно поэтому настоящее практическое занятие построено так, чтобы студент поэтапно реализовал учебный pipeline, а не просто выполнил несколько разрозненных действий над таблицей.

Работа выполняется по индивидуальному варианту. Для каждого варианта подготовлены наборы данных, согласованные по тематике с ПЗ 3.3.1 (ПЗ-9). Это позволяет выстроить последовательную методическую траекторию: сначала студент проектирует архитектуру системы, а затем на основе той же предметной области реализует её учебный вычислительный контур.

2. Цель работы

Цель практического занятия состоит в том, чтобы научить студента реализовывать упрощённый offline/online-пайплайн обработки данных и представлять его как воспроизводимый, структурированный и методически осмысленный процесс.

Иначе говоря, студент должен не только обработать данные, но и показать, что понимает:

- как данные загружаются;
- как очищаются;
- как объединяются с другими источниками;
- как превращаются в итоговую витрину;
- как отличается пакетная обработка от событийной;
- почему важны повторяемость и логика автоматического запуска.

3. Задачи практического занятия

В ходе выполнения работы необходимо:

- изучить структуру исходных данных по своему варианту;
- оценить качество данных и выявить возможные проблемы;
- выполнить очистку и подготовку данных;
- реализовать интеграцию основного и справочного набора данных;

построить batch-пайплайн обработки;
сформировать итоговую аналитическую витрину;
смоделировать упрощённую event-like обработку;
сравнить batch- и event-like подходы;
описать логику оркестрации и автоматизации;
сделать выводы о практическом смысле реализованного пайплайна.

4. Ожидаемые результаты

По итогам выполнения практического занятия студент должен:
понимать структуру учебного пайплайна обработки данных;
уметь работать с несколькими связанными файлами данных;
уметь выделять этапы загрузки, очистки, преобразования, интеграции и агрегации;
уметь строить итоговую аналитическую витрину;
понимать различие между пакетной и событийной логикой обработки;
уметь объяснить, зачем системе нужны оркестрация и автоматизация.

5. Исходные материалы и организация файлов

Для выполнения работы используются три типа файлов по каждому варианту.

Первый файл – основной набор данных:

variant_XX_main.csv

Второй файл – справочный набор данных, необходимый для объединения и обогащения основного набора:

variant_XX_reference.csv

Третий файл – набор для event-like обработки, который используется для имитации последовательного поступления событий:

variant_XX_stream.csv

Крайне важно, чтобы ноутбук и соответствующие ему CSV-файлы находились в одной папке. Шаблон ноутбука изначально настроен именно на такую организацию работы.

Студент должен использовать:

1. ноутбук-шаблон ПЗ 3.3.2;
2. архив датасетов;
3. при необходимости файл variant_column_mapping.csv, где указаны рекомендуемые значения служебных переменных time_col, entity_id_col, category_col, value_col, status_col, join_key.

6. Краткие теоретические сведения

6.1. Что такое pipeline

Pipeline, или пайплайн обработки данных, – это организованная последовательность этапов, в которой результат одного шага становится входом для следующего. В условиях реальных информационных систем именно пайплайн обеспечивает не разовое вычисление, а повторяемую обработку данных.

В данной работе пайплайн реализуется в упрощённой учебной форме, однако сохраняет промышленную логику. Он включает:

- загрузку данных;
- проверку качества;
- очистку;
- интеграцию;
- построение признаков;
- batch-агрегацию;
- получение витрины;
- упрощённую event-like обработку;
- описание оркестрации.

6.2. Что такое batch-обработка

Batch-processing, или пакетная обработка, – это обработка уже накопленного массива данных. Например, если у нас есть события за месяц, мы можем обработать их единым циклом, получить сводные показатели и построить отчёт. Такой подход удобен для аналитики, отчётности и формирования витрин.

6.3. Что такое event-like обработка

Event-like обработка в рамках данного практического занятия – это учебная имитация потокового режима, при котором данные рассматриваются как поступающие последовательно во времени. Полноценная потоковая платформа здесь не используется, но студент должен понять саму логику: в одном случае мы имеем доступ ко всему массиву данных сразу, в другом – обновляем показатели по мере поступления событий.

6.4. Что такое аналитическая витрина

Аналитическая витрина – это итоговая таблица или компактный аналитический результат, пригодный для использования пользователем, аналитиком, руководителем или внешней системой. Важно понимать, что конечной целью пайплайна является не промежуточный dataframe, а именно результат для потребителя.

6.5. Что такое оркестрация

Оркестрация – это управление порядком выполнения задач. В данной работе оркестрация описывается концептуально: студент показывает, какие этапы следуют друг за другом, какие зависят от предыдущих и что нужно делать при ошибках. Даже в учебной среде это важно, потому что без такого понимания Jupyter Notebook превращается в набор случайных ячеек.

7. Программные средства

Работа выполняется в среде Python с использованием Jupyter Notebook. Основные библиотеки уже подключены в шаблоне:

- pandas – для обработки таблиц;
- pumpru – для числовых операций;
- matplotlib – для построения графиков;
- pathlib – для работы с путями к файлам.

Специально устанавливать дополнительные библиотеки для базового варианта работы не требуется.

8. Пошаговый порядок выполнения работы

Шаг 1. Подготовка рабочей папки

Создайте отдельную папку для своего варианта. В неё необходимо поместить:

ноутбук PZ_3_3_2_Data_Pipeline_Template.ipynb;

три файла данных по своему варианту;

при необходимости – файл variant_column_mapping.csv.

После этого откройте ноутбук и убедитесь, что он и данные действительно лежат в одной директории.

Шаг 2. Выбор варианта

В одной из первых ячеек ноутбука находится переменная: `variant_number = 1`

Студент должен заменить значение 1 на номер своего варианта. После запуска ячейки появится название предметной области и ожидаемые имена файлов. Это первый контрольный шаг: если названия совпадают с имеющимися файлами, можно двигаться дальше.

Шаг 3. Постановка задачи

В markdown-разделе ноутбука нужно заполнить:
формулировку варианта;
краткое описание предметной области;
цель пайплайна.

На этом этапе нельзя писать общие фразы вроде «обработать данные». Нужно указать, какой именно аналитический результат должен быть получен. Например: построить витрину задержек доставки, витрину активности пользователей, витрину аномальных событий, витрину нагрузки на службу поддержки и так далее.

Шаг 4. Загрузка исходных данных

После выбора варианта запускается ячейка загрузки данных. Она читает:
основной файл;
справочный файл;
stream-файл.

На этом этапе студент должен:
посмотреть на первые строки таблиц;
оценить размерности наборов данных;
понять, какие сущности представлены в файлах;
определить, какие таблицы являются основными, а какие вспомогательными.

Сразу после этого следует сделать краткий текстовый вывод: что за данные загружены, какую роль они играют и какие поля кажутся ключевыми.

Шаг 5. Первичный анализ качества данных

Далее необходимо выполнить первичную диагностику качества. Для этого используются:
подсчёт пропусков;
подсчёт дубликатов;
вывод типов столбцов;

просмотр описательной статистики;
анализ категориальных признаков.

На данном этапе важно не просто выполнить код, а понять проблему данных. Например:

есть ли пропущенные временные метки;
есть ли повторяющиеся события;
не записано ли время как обычная строка;
нет ли аномально больших или отрицательных числовых значений.

После анализа необходимо письменно указать, какие проблемы качества данных обнаружены и почему они мешают дальнейшей обработке.

Шаг 6. Настройка ролей столбцов

В шаблоне ноутбука есть отдельная ячейка, где студент должен указать, какие столбцы выполняют роль:

времени;
идентификатора объекта;
категории;
числового значения;
статуса;
ключа для объединения.

Это один из важнейших этапов, потому что именно от правильной настройки этих переменных зависит корректность всей дальнейшей обработки. Чтобы не ошибиться, нужно использовать файл `variant_column_mapping.csv`.

Если какой-то тип столбца отсутствует в конкретном варианте, допускается оставить значение `None`, но это следует делать осознанно.

Шаг 7. Очистка данных

На следующем этапе вызывается функция `clean_data()`. Она выполняет: удаление дубликатов;
нормализацию строковых значений;
преобразование временного поля в формат `datetime`.

Студент должен не просто принять результат, а проверить: уменьшилось ли количество дубликатов;
не пропали ли важные записи;
корректно ли преобразовалось время;
не появились ли значения `NaT` после преобразования временных столбцов.

После очистки следует сделать текстовый вывод: какие именно проблемы были устранены и почему этап очистки нельзя пропускать.

Шаг 8. Интеграция источников

Если в варианте имеется справочный набор данных, его необходимо объединить с основным набором по ключевому полю. Это делается через `merge`.

На этом этапе студент должен понимать:
какой столбец используется как ключ;
почему выбран именно этот ключ;

как изменилось число столбцов и строк после объединения;
не появились ли лишние пропуски или дублирование после join.

Если интеграция в конкретном варианте не требуется, это тоже нужно явно написать и объяснить.

Шаг 9. Преобразование и построение признаков

После интеграции выполняется построение производных признаков.

Например:

дата без времени;

час события;

день недели;

признак выходного дня.

В некоторых вариантах студент может добавить и собственные производные признаки, если они помогают интерпретировать данные. Например: признак просрочки; интервал обработки; укрупнённую категорию событий.

Главное требование – каждый созданный признак должен быть осмыслен. Нельзя создавать столбцы только ради увеличения их числа.

Шаг 10. Построение batch-пайплайна

Это центральный этап работы. Здесь данные проходят через функции: загрузка; подготовка; преобразование; агрегация; формирование витрины.

Студент должен запустить batch-пайплайн и получить итоговую таблицу batch_report. В зависимости от варианта это может быть: число событий по датам и типам; сумма значений по категориям; средний показатель по временным интервалам; распределение обращений, продаж, инцидентов, заказов и т.д.

Очень важно понимать, что batch_report – это не просто технический результат, а уже аналитическая витрина.

После этого следует кратко описать: какие этапы выполнил batch-пайплайн; что именно находится в итоговой витрине; почему эта витрина полезна пользователю.

Шаг 11. Сохранение batch-витрины

В ноутбуке предусмотрена ячейка для сохранения итоговой витрины в CSV-файл. Её необходимо выполнить. Это помогает показать, что пайплайн должен не только считать показатели, но и уметь формировать пригодный для передачи результат.

Шаг 12. Визуализация результата

Далее строится 1–3 осмысленных графика. В типовом варианте это может быть:

динамика количества событий по датам;
распределение по категориям;
график среднего значения по временным периодам.

После построения графика обязательно нужно написать интерпретацию. Недостаточно просто показать рисунок. Нужно пояснить: что именно видно на графике; какие особенности или закономерности можно заметить; как это связано с предметной областью варианта.

Шаг 13. Event-like обработка

После batch-пайплайна выполняется event-like этап. Для этого используется stream-файл либо, если его нет, основной набор данных.

Смысл этого раздела – показать, как выглядела бы работа системы, если бы данные поступали последовательно. В шаблоне ноутбука реализован простой накопительный и скользящий расчёт.

Студент должен обратить внимание:

что данные предварительно сортируются по времени;
что показатели обновляются не по всему массиву сразу, а по мере движения по событиям;
что event-like логика лучше подходит для оперативного наблюдения, но менее удобна для полной отчётности.

После этого этапа необходимо письменно сравнить event-like и batch-подходы.

Шаг 14. Сравнение batch и event-like обработки

В ноутбуке есть таблица сравнения. Она должна быть заполнена осмысленно. Необходимо сопоставить два подхода по следующим критериям:

полнота данных;
скорость получения результата;
удобство агрегации;
пригодность для отчётности;
пригодность для оперативного реагирования.

На этом этапе студент должен сформулировать важный вывод: разные режимы обработки данных нужны для разных целей, и один не заменяет другой.

Шаг 15. Описание оркестрации

Далее необходимо заполнить таблицу оркестрации. Студент должен показать:

какие этапы выполняются по порядку;
какие зависят друг от друга;
какие можно запускать по расписанию;
что делать, если один из этапов не выполнен.

Особое внимание следует уделить ответу на вопрос: почему ручной запуск неудобен. Здесь нужно показать понимание того, что в реальной системе регулярные расчёты должны быть автоматизированы.

Шаг 16. Схема учебного пайплайна

В конце ноутбука студент должен вставить схему. Она может быть построена в любом удобном редакторе и затем вставлена в notebook как изображение.

На схеме должны быть отражены:

входные данные;
очистка;
интеграция;
построение признаков;
batch-агрегация;
event-like обработка;
витрина;
условный блок оркестрации.

Схема нужна для того, чтобы студент связал код с архитектурным мышлением.

Шаг 17. Итоговые выводы

В заключительном разделе ноутбука студент формулирует выводы. В них необходимо отразить:

какой пайплайн был реализован;
какие этапы оказались обязательными;
что показала batch-обработка;
что показала event-like обработка;
какой аналитический результат получен;
почему этот notebook можно считать учебной моделью промышленного пайплайна.

9. Требования к оформлению результата

Работа считается выполненной, если студент представил:

заполненный ноутбук;
выполненный код по всем обязательным разделам;
итоговую batch-витрину;
минимум одну визуализацию;
раздел event-like обработки;
заполненную таблицу сравнения batch и event-like режимов;
таблицу оркестрации;
схему пайплайна;
развёрнутые выводы.

Ноутбук должен быть структурированным. Недопустимо сдавать набор ячеек без заголовков, комментариев и промежуточных выводов.

10. Типичные ошибки студентов

Наиболее распространённые ошибки при выполнении ПЗ 3.3.2 следующие.

Первая ошибка – студент не настраивает роли столбцов и запускает шаблон, не подогнав его под свой вариант. В результате временное поле, категория или ключ интеграции не определяются корректно.

Вторая ошибка – очистка данных выполняется формально, без анализа результата. Например, дубликаты удалены, но никто не проверил, не остались ли они, а временное поле преобразовано, но появившиеся пустые значения не осмыслены.

Третья ошибка – batch-пайплайн не оформлен как процесс. Студент делает несколько вычислений, но не показывает их последовательность и назначение.

Четвёртая ошибка – event-like обработка воспринимается как лишняя формальность. На самом деле именно она помогает понять разницу между отчётным и оперативным режимами аналитики.

Пятая ошибка – визуализация строится без интерпретации. Для методической полноты каждый график должен сопровождаться объяснением.

Шестая ошибка – в разделе оркестрации студент ограничивается общими словами и не показывает зависимости между этапами.

11. Контрольные вопросы для самопроверки

Перед сдачей работы студент должен уметь ответить на следующие вопросы.

Что такое batch-пайплайн?

Почему этап очистки является обязательным?

Что такое аналитическая витрина?

Чем event-like обработка отличается от пакетной?

Почему итоговый результат должен быть ориентирован на пользователя?

Зачем системе нужна оркестрация?

Почему Jupyter Notebook в этой работе рассматривается как модель пайплайна, а не просто как среда программирования?

12. Критерии оценивания

Работа оценивается по следующим группам критериев.

Первая группа – корректность обработки данных. Оценивается правильность загрузки, очистки, интеграции и построения признаков.

Вторая группа – структурность пайплайна. Оценивается, насколько ясно студент выделил этапы и связал их в единый процесс.

Третья группа – качество batch-результата. Оценивается содержательность итоговой витрины, её пригодность для анализа и интерпретации.

Четвёртая группа – понимание event-like обработки. Оценивается, увидел ли студент различие между двумя режимами работы с данными.

Пятая группа – качество методического оформления. Оцениваются текстовые выводы, интерпретация графиков, описание оркестрации и финальные выводы.

13. Рекомендуемая структура итогового вывода

Чтобы студенту было проще завершить работу, можно рекомендовать такой порядок построения заключения.

Сначала кратко указать, какая предметная задача решалась.
Затем описать, какие источники данных использовались.
После этого перечислить ключевые этапы пайплайна.
Далее пояснить, какой итоговый аналитический результат был получен.
Затем сравнить batch- и event-like обработку.
И, наконец, сформулировать, в чём состоит практический смысл построенного учебного пайплайна.

14. Методический смысл работы

Главный смысл ПЗ 3.3.2 заключается в том, чтобы студент начал воспринимать анализ данных как систему, а не как набор локальных вычислений. Даже если используемые данные синтетические, а event-like обработка упрощена, сама логика работы остаётся профессионально важной. Именно она подготавливает переход от учебных задач к реальным Big Data-системам.